

03 JS var let const

Complete Guide: var vs let vs const



Theory Section

What are Variables?

Variables are containers that store data values. Think of them as labeled boxes where you can put information and retrieve it later.

The Three Keywords: var, let, const

Keyword	Scope	Reassignable	Redeclarable	Hoisting Behavior
var	Function	✓ Yes	✓ Yes	Hoisted (initialized as undefined)
let	Block	✓ Yes	✗ No	Hoisted (not initialized - TDZ)
const	Block	✗ No	✗ No	Hoisted (not initialized - TDZ)



Lab Setup

Step 1: Create Your Project Structure

1. Create a new folder called `js-variables-lab`
2. Inside it, create two files:
 - `index.html`
 - `script.js`

Step 2: Setup HTML File

Open `index.html` and paste this code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>var vs let vs const Lab</title>
</head>
<body>
  <h1>🔗 JavaScript Variables Lab</h1>
  <div class="info">
    <p>Open your browser's <strong>Console</strong> (F12 or Right-click → Inspect → Console)</p>
    <p>Watch the console as you follow each experiment!</p>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

Step 3: Start Live Server

1. Right-click on `index.html` in VS Code
2. Select "Open with Live Server"
3. Open Browser Console (F12 or Right-click → Inspect → Console tab)



Experiment 1: Understanding var

Theory

- `var` has **function scope** or **global scope**
- Can be **reassigned** and **redeclared**
- Gets **hoisted** to the top of its scope

Lab Exercise 1A: Basic var Usage

Open `script.js` and add:

JS

```
console.log("=== EXPERIMENT 1A: Basic var ===");

var name = "Alice";
console.log(name); // Output: Alice

var name = "Bob"; // Redeclaring is allowed
console.log(name); // Output: Bob

name = "Charlie"; // Reassigning is allowed
console.log(name); // Output: Charlie
```

Save and check your console! You should see all three names printed.

Lab Exercise 1B: var and Function Scope

Add this to your script.js:

JS

```
console.log("=== EXPERIMENT 1B: var Function Scope ===");

function testVar() {
  var age = 25;
  console.log("Inside function:", age); // Works fine
}

testVar();
// console.log("Outside function:", age); // Uncomment this line - it will cause an
error!
```

What happened? The `age` variable only exists inside the function. Uncomment the last line to see the error!

Lab Exercise 1C: var Has NO Block Scope

Add this code:

JS

```
console.log("=== EXPERIMENT 1C: var and Blocks ===");

if (true) {
  var city = "New York";
  console.log("Inside block:", city); // Output: New York
}

console.log("Outside block:", city); // Output: New York - var leaks out!
```

Key Learning: `var` ignores block boundaries (if, for, while blocks). It leaks out!

Lab Exercise 1D: var Hoisting

Add this code:

```
JS
console.log("=== EXPERIMENT 1D: var Hoisting ===");

console.log(greeting); // Output: undefined (not an error!)
var greeting = "Hello";
console.log(greeting); // Output: Hello
```

What's happening? JavaScript moves `var` declarations to the top:

```
JS
// Behind the scenes, JavaScript does this:
var greeting; // Declaration hoisted
console.log(greeting); // undefined
greeting = "Hello"; // Assignment stays here
console.log(greeting); // Hello
```



Experiment 2: Understanding let

Theory

- `let` has **block scope** (respects {})
- Can be **reassigned** but NOT **redeclared**
- Hoisted but in **Temporal Dead Zone (TDZ)**

Lab Exercise 2A: Basic let Usage

Add to script.js:

```
JS
console.log("=== EXPERIMENT 2A: Basic let ===");

let fruit = "Apple";
console.log(fruit); // Output: Apple

fruit = "Banana"; // Reassigning works
console.log(fruit); // Output: Banana

// let fruit = "Orange"; // Uncomment this - ERROR! Cannot redeclare
```

Try uncommenting the last line! You'll see: `SyntaxError: Identifier 'fruit' has already been declared`

Lab Exercise 2B: let Has Block Scope

Add this code:

```
JS
console.log("=== EXPERIMENT 2B: let Block Scope ===");

if (true) {
  let color = "Blue";
  console.log("Inside block:", color); // Output: Blue
}

// console.log("Outside block:", color); // Uncomment - ERROR! color is not defined
```

Key Learning: `let` respects block boundaries. It stays inside the {}!

Lab Exercise 2C: let in Loops (The Classic Example)

Add this code:

```
JS
console.log("=== EXPERIMENT 2C: let in Loops ===");

// Using let (correct behavior)
for (let i = 0; i < 3; i++) {
  setTimeout(() => {
    console.log("let loop:", i); // Output: 0, 1, 2
  }, 100);
}

// Using var (unexpected behavior)
for (var j = 0; j < 3; j++) {
  setTimeout(() => {
    console.log("var loop:", j); // Output: 3, 3, 3
  }, 100);
}
```

Wait for it... After a brief delay, see the difference! `let` creates a new variable for each iteration, but `var` doesn't.

Lab Exercise 2D: Temporal Dead Zone (TDZ)

Add this code:

JS

```
console.log("≡ EXPERIMENT 2D: let and TDZ ≡");

// console.log(movie); // Uncomment - ERROR! Cannot access before initialization
let movie = "Inception";
console.log(movie); // Output: Inception
```

Key Learning: Unlike `var`, you CANNOT access `let` variables before declaration!

The Temporal Dead Zone (TDZ) in JavaScript refers to the period during which `let` and `const` variables exist within their scope but are not yet initialized. Attempting to access these variables during this period will result in a `ReferenceError`.

Here's a breakdown:

- **Hoisting and TDZ:** While `let` and `const` declarations are "hoisted" to the top of their block scope, unlike `var`, they are not initialized with `undefined`. Instead, they enter the TDZ.
- **Start of TDZ:** The TDZ begins at the start of the block scope where a `let` or `const` variable is declared.
- **End of TDZ:** The TDZ ends when the variable is declared and initialized with a value.
- **Consequences of Accessing in TDZ:** If you try to use a `let` or `const` variable before its declaration and initialization within its scope, JavaScript will throw a `ReferenceError`.

Example:

JS

```
function exampleTDZ() {
  // 'myVariable' is in the TDZ from this point until its declaration
  console.log(myVariable); // ReferenceError: Cannot access 'myVariable' before
  initialization

  let myVariable = "Hello"; // TDZ ends here, and 'myVariable' is initialized
  console.log(myVariable); // Outputs: Hello
}

exampleTDZ();
```

Why the TDZ?

The TDZ was introduced with ES6 (ECMAScript 2015) to provide more predictable and safer variable handling compared to `var`. It prevents the use of variables before they have a defined value, which can help catch potential bugs and encourage better coding practices.

<https://www.c-sharpcorner.com/article/understanding-the-temporal-dead-zone-tdz-in-javascript/>



Experiment 3: Understanding const

Theory

- **const** has **block scope** (like let)
- CANNOT be **reassigned** or **redeclared**
- Must be **initialized** when declared
- Objects/Arrays: reference is constant, but content can change

Lab Exercise 3A: Basic const

Add to script.js:

```
console.log("=== EXPERIMENT 3A: Basic const ===");

const PI = 3.14159;
console.log(PI); // Output: 3.14159

// PI = 3.14; // Uncomment - ERROR! Assignment to constant variable
// const GRAVITY; // Uncomment - ERROR! Missing initializer
```

JS

Try uncommenting! You'll see errors because **const** is strict.

Lab Exercise 3B: const with Objects (Important!)

Add this code:

```
console.log("=== EXPERIMENT 3B: const with Objects ===");

const person = {
  name: "John",
  age: 30
};

console.log("Original:", person);

// Modifying properties is ALLOWED
person.age = 31;
person.city = "Boston";
console.log("Modified:", person);

// But reassigning is NOT allowed
// person = { name: "Jane" }; // Uncomment - ERROR!
```

JS

Key Learning: **const** protects the reference, not the content!

Lab Exercise 3C: const with Arrays

Add this code:

```
console.log("=== EXPERIMENT 3C: const with Arrays ===");

const numbers = [1, 2, 3];
console.log("Original:", numbers);

// Modifying array is ALLOWED
numbers.push(4);
numbers[0] = 10;
console.log("Modified:", numbers);

// But reassigning is NOT allowed
// numbers = [5, 6, 7]; // Uncomment - ERROR!
```

JS

Lab Exercise 3D: const Block Scope

Add this code:

```
console.log("=== EXPERIMENT 3D: const Block Scope ===");

const country = "USA";

if (true) {
    const country = "Canada"; // Different variable!
    console.log("Inside block:", country); // Output: Canada
}

console.log("Outside block:", country); // Output: USA
```

JS

Experiment 4: Side-by-Side Comparison

Add this final comprehensive example:


```
console.log("≡ EXPERIMENT 4: Complete Comparison ≡");

// Scenario: Loop with button clicks simulation
console.log("Creating 3 buttons with different variable types:");

// Using var (problematic)
for (var x = 1; x ≤ 3; x++) {
  setTimeout(() ⇒ {
    console.log("Button", x, "clicked (var)"); // All show 4!
  }, 1000);
}

// Using let (correct)
for (let y = 1; y ≤ 3; y++) {
  setTimeout(() ⇒ {
    console.log("Button", y, "clicked (let)"); // Shows 1, 2, 3
  }, 1000);
}

// Using const in forEach (modern approach)
[1, 2, 3].forEach((z) ⇒ {
  setTimeout(() ⇒ {
    console.log("Button", z, "clicked (const)"); // Shows 1, 2, 3
  }, 1000);
});
```



Best Practices & Quick Reference

When to Use What?

JS

```
// ❌ AVOID var - it's old and problematic  
var oldWay = "Don't use this";
```

```
// ✅ USE const by default (most common)  
const API_KEY = "abc123";  
const user = { name: "Alice" };  
const items = [1, 2, 3];
```

```
// ✅ USE let when you need to reassign  
let counter = 0;  
counter++;
```

```
let status = "pending";  
status = "complete";
```

Decision Tree

Do you need to reassign the variable?

|

├ NO → Use const ✅

|

└ YES → Use let ✅

Never use var ❌

Common Mistakes to Avoid

JS

```

console.log("=== COMMON MISTAKES ===");

// Mistake 1: Using var in loops
console.log("Don't do this:");
for (var i = 0; i < 3; i++) {
    // var leaks out of the loop
}
console.log(i); // i is still accessible! Bad!

// Correct:
for (let i = 0; i < 3; i++) {
    // let is contained in the loop
}
// console.log(i); // Error - i is not defined. Good!

// Mistake 2: Not initializing const
// const value; // ERROR!
const value = 10; // Correct

// Mistake 3: Trying to reassign const primitives
const age = 25;
// age = 26; // ERROR!

// But this is OK:
const person2 = { age: 25 };
person2.age = 26; // OK - modifying property, not reassigning

```



Summary Table

Feature	var	let	const
Scope	Function/Global	Block	Block
Reassign	✓	✓	✗
Redeclare	✓	✗	✗
Hoisting	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Use in 2025?	✗ No	✓ Sometimes	✓ Default



Challenge Exercise

Add this to practice everything you learned:

JS

```
console.log("=== CHALLENGE: Fix the Code ===");

// This code has problems. Can you identify them?
function calculateTotal() {
  var items = [10, 20, 30];
  var total = 0;

  for (var i = 0; i < items.length; i++) {
    var item = items[i];
    total = total + item;
  }

  return total;
}

console.log("Total:", calculateTotal());

// Now rewrite it using let and const appropriately!
// Your solution here:
function calculateTotalModern() {
  const items = [10, 20, 30];
  let total = 0;

  for (let i = 0; i < items.length; i++) {
    const item = items[i];
    total = total + item;
  }

  return total;
}

console.log("Modern Total:", calculateTotalModern());
```



Congratulations!

You've completed the var vs let vs const lab! Key takeaways:

1. **Always use `const` by default** - it prevents accidental reassignment
2. **Use `let` when you need to reassign** - like counters or status values
3. **Never use `var`** - it's outdated and causes scope issues

4. **Remember:** `const` objects/arrays can have their contents modified

Keep this guide handy as a reference, and happy coding! 🚀